Pre-Reads: Functions In Python

Nishut Suman 10 Jan, 2026 At 10:00 AM Pre-Reads Foundation Recommended Resource

Associated Lectures (1)

Description Discussions

Pre-Read: Python Functions – The Building Blocks of Smarter Code

Think of functions as your personal assistants in Python.

You give them a task once, and they'll handle it every time you ask, no repetition, no fuss.

What's a Function?

A **function** is a reusable block of code that runs only when you call it.

It can even return something back to you in the way a calculator giving you the result.

```
def greet():
    print("Hello, This is my first Python function")
```

When you call greet(), it prints the message.

The indented lines show the function's **body, so remember** indentation matters in Python!

Calling a Function

Once defined, simply **call** it by name with parentheses:

```
def my_function():
    print("Hello from a function")

my_function()
```

Output:

```
Hello from a function
```

That’s it ! your function just ran!

Naming Your Function

Follow the same rules as variable names:

- Start with a letter or underscore (_)
- Use only letters, numbers, or underscores
- Be **case-sensitive**

Examples:

```
calculate_sum()
_private_function()
myFunction2()
```

Tip: Pick names that describe what the function does.

How Functions Receive Data (Arguments)

Functions can take **input values**, known as **arguments**.

There are two common ways to pass them:

Positional arguments

Keyword arguments

---

1. Positional Arguments

You pass values **in order**, matching each parameter.

```
def add(a, b):
    return a + b

print(add(5, 10))
```

Output:

```
15
```

Remember: The order matters, so swap them, and the result may change!

---

2. Keyword Arguments

Instead of relying on order, you use **names** of parameters.

```
def nameAge(name, age):
    print("Hi, I am", name)
    print("My age is", age)

nameAge(name="Prince", age=20)
nameAge(age=20, name="Prince")
```

Output:

```
Hi, I am Prince
My age is 20
Hi, I am Prince
My age is 20
```

Keyword arguments make your code clearer and less error-prone.

Keyword vs Positional: A Quick Glance

Keyword Arguments	Positional Arguments
Use parameter names	Follow parameter order
Order doesn't matter	Order matters
More readable	More concise
`func(a=10, b=20)`	`func(10, 20)`

Returning Values

Sometimes you don't just want to print, you want the function to *give back* a result.

That's where **return** comes in.

```
def multiply(x):
    return 5 * x

print(multiply(3))
print(multiply(5))
print(multiply(9))
```

Output:

```
15
25
45
```

Once Python hits return, it stops the function and sends the value back.

Local vs Global Variables

Python keeps track of where each variable lives — **inside** or **outside** a function.

Local Variables

Live **inside** the function — can’t be used elsewhere.

```
def greet():
    msg = "Hello from inside the function!"
    print(msg)

greet()
```

Output:

```
Hello from inside the function!
```

Global Variables

Defined **outside** any function and can be used anywhere.

```
msg = "Python is awesome!"

def display():
```

```
    print("Inside function:", msg)

display()
print("Outside function:", msg)
```

Output:

```
Inside function: Python is awesome!
Outside function: Python is awesome!
```

Quick Recap

- Functions help you **reuse code** and keep it **clean**.
- Define with def, call with ().
- Pass data using **positional** or **keyword** arguments.
- Use **return** to get results back.
- Know where your variables live (**local** vs **global)**.

Pro Tip:

Play around with your own examples, tweak the names, change arguments, or add returns.

The more you practice writing functions, the more natural it feels.